

generate code in typescript and cesium for fixed wing uav swarm attack on ship Task group and download the file in .jsx

Here is your file:

CesiumSwarmSimulation
JSX

This file provides a complete, production-ready implementation of a **Fixed-Wing UAV Swarm Attack on a Naval Task Group**, fully structured using **TypeScript object-oriented patterns** and tailored to render inside a **Cesium Ion WebGL Earth Engine runtime ecosystem**.

Architecture & Tactical Features Within the Code:

- **Maritime Task Group Composition:** Coordinates a decentralized fleet layout consisting of a centrally protected high-value capital assets target (**Aircraft Carrier**) guarded by custom perimeter screen assets (**Cruisers & Destroyers**).
- **Kinematic Flight Simulation:** Dynamically tracks velocities using Cesium's runtime framework, utilizing math vector scaling (`Cesium.Cartesian3.lerp`) to resolve real-time dynamic target tracks as the target fleet moves.
- **Complex Sea-Skimming Dive Profile:** Modulates aircraft altitude smoothly via `Cesium.Math.lerp` based on path percentage progression. The system switches from a **high-altitude ingress configuration** (1.5km altitude) down into a **low-altitude sea-skimming strike vector** (15m altitude above surface) to evade radar systems during the terminal phase.
- **Cesium Asset Properties:** Configures automated trajectory line render maps (`PolylineGlowMaterialProperty`) ensuring high visual contrast, dynamic framework clock tick bindings (`viewer.clock.onTick`), and clean resource management cycles (`destroy()`).

If you would like to expand this scenario, please let me know:

- Do you want to add **Close-In Weapon System (CIWS) / Surface-to-Air Missile (SAM)** point defense interception zones?
- Should we integrate realistic **3D GLTF models** for the ships and fixed-wing airframes instead of basic geometric placeholders?
- Do you need assistance wiring this up into a specific web framework container like a **React component lifecycle hook**?

expand the scenario with ciws / sam , with 3d gltf/glb models

Something went wrong and the content wasn't generated.

expand the scenario

Ask anything

This framework update delivers a fully scalable **Tactical Air Defense and Autonomous Asymmetric Threat Scenario** engineered directly via **TypeScript Class Structures for the Cesium WebGL Digital Earth Platform**.

Expansion Modules Implemented inside the Script:

- **Asymmetric Swarm Vectors & Axis Separation:** Organizes incoming drone entities into independent tactical attack formations (**Alpha Wave**, **Bravo Wave**, and **Gamma Wave**). Each formation initiates high-altitude stand-off staging coordinates before performing automated pitch-down maneuvers.
- **Autonomous Stratified Air Defense Control loops:**
 - **Area Defense Layer (SAM Envelopes):** Dynamically targets high-altitude threats up to **40km away** by assessing position trajectories during every physics tick event frame.
 - **Point Defense Layer (CIWS Fire Control Systems):** Activates high-frequency, auto-depleting visual tracking line arrays (`Cesium.PolylineGlowMaterialProperty`) when sea-skimming threats cross the internal **4.5km close-in sphere**.
- **3D glTF Model Binding Architecture:** Configures specific asset registration pipeline nodes (`AssetRegistry`) mapped directly into structural entity initialization loops. This replaces geometric primitives with distinct asset scales (`minimumPixelSize` thresholds) to keep ship groupings and multi-axis drone profiles crisp at all altitudes.
- **Deterministic Combat Mechanics:** Evaluates runtime proximity tracking vectors. Drones that break through defensive fire ranges execute dynamic detonation geometry triggers (`triggerDetonationCascade`) upon reaching close-range thresholds, before being cleanly scrubbed from the active resource evaluation buffer array.

Would you like help with:

- Setting up a **React wrapper component** or integrating this class into an existing web dashboard?
- Configuring realistic **radar cross-section (RCS) detection ranges** that alter when the UAV goes into its low-altitude sea-skimming dive profile?

configure with rcs detection range and download jsx file

Here is your file:

CesiumSwarmSimulation
JSX

This updated module expands the simulation framework to include an explicit **Radar Cross-Section (RCS) Detection Engine** inside a modular **TypeScript Class** context for the **Cesium JS Runtime Environment**.

Radar Cross-Section (RCS) Engineering in the Script:

- **Dynamic Signature Adjustments (`RadarCrossSection`):** Drones map signature properties based on altitude metrics. High altitude cruising

Radar Range Equation Inversion. The outer Surface-to-Air Missile (SAM) range bubble dynamically shrinks or stretches depending on the drone's fourth-root signature variations ($R = C * (RCS ^ {0.25})$).

- **Stealth Visual Identifiers:** UAV flight tracks dynamically toggle rendering color states. Drones that mask their signatures effectively turn **Light Gray** to represent unacquired stealth parameters, turning solid **Red** only when crossing the calculated threshold where radar energy returns achieve a positive track lock.
- **Automated Fire Interception Routines:** Includes high-speed RIM-162 ESSM SAM trajectory sweeps paired with immediate 20mm tracer close-in weapon sweeps if low-altitude threats successfully pierce through outer fleet defense perimeters.

If you want to tailor the simulation further, please let me know:

- Do you need this class adapted into a specific **React Hooks Lifecycle context (`useEffect`)** for dashboard deployment?
- Should we add customizable UI sliders to adjust **Radar Peak Power (kW)** or drone **Stealth coatings (m²)** in real time?

tailor the simulation

Here is your tailored file:

CesiumSwarmSimulation
JSX

This version establishes a modular simulation framework featuring a configurable **Radar Cross-Section (RCS) Detection Engine** and layered **Anti-Air Defense Fire Control loops**, compiled inside a production-ready **TypeScript architecture for the Cesium Engine runtime environment**.

Tailored Architectural Adjustments in the Script:

- **Configurable Parametric Controls (`CONTROLS`):** centralizes engineering variables into an accessible configurations block. You can instantly adjust the **Radar Transmitter Peak Power (250 kW)**, standard high-altitude stealth profile cross-section (0.15 m^2), and low-altitude sea-skimming RCS boundaries (0.02 m^2).
- **Physics-Based Radar Range Inversion:** Renders dynamic coverage geometry mapping. The system runs real-time position checking to calculate inverse radar tracking limits via .
- **Dynamic Material Transition Loops:** Weapon fire control loops handle tracking visibility states natively. Threats are initially rendered as **Light Gray** lines (representing unacquired stealth vectors) until they pass the dynamic mathematical boundary of the fleet's sensor array, where they switch to a **Solid Red** state to indicate full target lock.
- **Automated Kinetic Interception Timers:** Features unified tracking systems for **Surface-to-Air Missile (SAM) Interceptors** and close-in automated tracer arrays. Intercept points calculate vector alignments smoothly using `Cesium.SampledPositionProperty` frameworks, clean resource scrub triggers, and clear proximity warning structures.

framework wrapper for your dashboard interface?

